

4. Software Development

4.1. Program Development Cycle

- **Analyze problem:** define problem, record program specifications and recognize inputs, process, output & UI
- **Design program:** develop logic plan, write algorithm in e.g. pseudocode or flowchart and test solution
- **Code program:** translate algorithm into high level language with comments/remarks and produce user interface with executable processes
- **Test and debug program:** test program using test data, find and correct any errors and ensure results are correct
- **Formalize solution:** review program code, revise internal documentation and create end-user documentation
- **Maintain program:** provide education and support to end-user, correct any bugs and modify if user requests

There are three different development life cycles:

- **Waterfall model:** a classical model, used to create a system with a linear approach, from one stage to another
- **Iterative model:** a initial representation starts with a small subset, which becomes more complex over time until the system is complete
- **Rapid Application Development (RAD) model:** a prototyping model, with no (or less) specific planning put into it. More emphasis on development and producing a product-prototype.

4.2. Integrated Development Environment

- A software application that allows the creation of a program e.g. Python
- Consists of a source code editor, build automation tools, a debugger

Coding:

- Reserved words are used by it as command prompts
- Listed in the end-user documentation of IDE
- A series of files consisting of preprogrammed-subroutines may also be provided by the IDE

Initial Error Detection:

- The IDE executes the code & initial error detection carried out by compiler/interpreter doing the following:
 - **Syntax/Logic Error:** before program is run, an error message warns the user about this
 - **Runtime Error:** run of the program ends in an error

Debugging:

- **Single stepping:** traces through each line of code and steps into procedures. Allows you to view the effect of each statement on variables

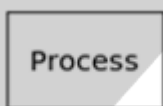
- Breakpoints: set within code; program stops temporarily to check that it is operating correctly up to that point
- Variable dumps (report window): at specific parts of program, variable values shown for comparison

4.3. Structure Charts

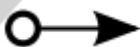
- **Purpose:** used in structured programming to arrange program modules, each module represented by a box
- Tree structure visualizes relationships between modules, showing data transfer between modules using arrows.
- Example of a top-down design where a problem (program) is broken into its components.

Rules:

- **Process:** Represents a programming module e.g. a calculation



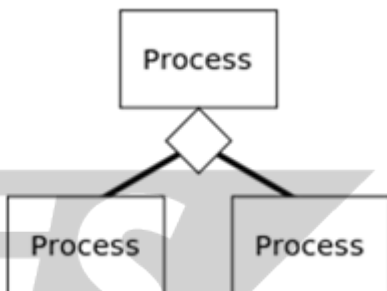
- **Data couple:** Data being passed from module to module that needs to be processed



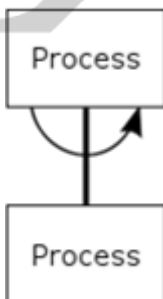
- **Flag:** Check data sent to start or stop a process. E.g. check if data sent in the correct format



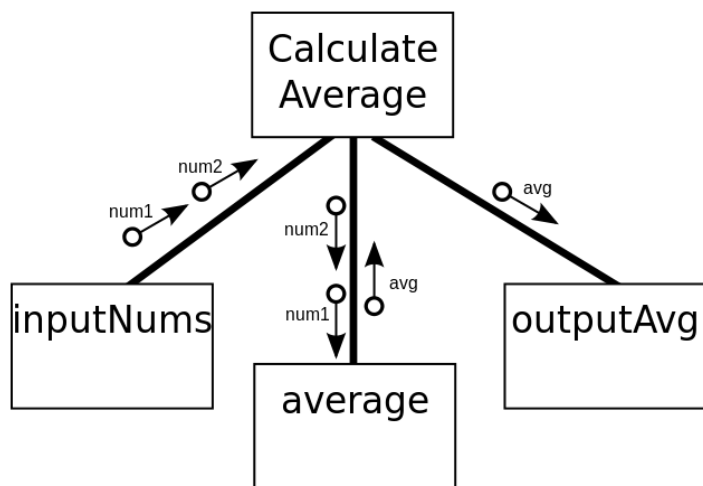
- **Selection:** Condition will be checked and depending on the result, different modules will be executed



- **Iteration:** Implies that module is executed multiple times



Example:



4.4. Types of Errors

Syntax errors:

- When source code does not obey rules of the language
- Compiler generates error messages
- Examples:
 - Misspell identifier when calling it
 - Missing punctuation – colon after if
 - Incorrectly using a built-in function
 - Argument being made does not match data type

Run-time errors:

- Source code compiles to machine code but fails upon execution (*red lines show up in Python*)
- When the program keeps running and you have to kill it manually
- Examples:
 - Division by 0
 - Infinite loop – will not produce error message, program will just not stop until forced to

Logic errors:

- Program works but gives incorrect output
- Examples:
 - Out By One – when '>' is used instead of '>='
 - Misuse of logic operators

4.5. Corrective Maintenance

- Corrective Maintenance is correcting identified errors
- **White-Box testing:** making sample data and running it through a trace table
- **Trace table:** technique used to test algorithms; make sure that no logical errors occur e.g.

Line	largest	x	value[x] > largest	output
1	38	-		
2		2		
3			false	

4.6. Adaptive Maintenance

- Making amendments to:
 - **Parameters:** due to changes in specification
 - **Logic:** to enhance functionality or more faster or both
 - **Design:** to make it more user friendly

4.7. Perfective Maintenance

- Making modifications or improvements to a system that is already **functioning correctly** to enhance its performance, efficiency, or maintainability. It is about optimizing what already works, not fixing errors.
- Driven by user feedback, code reviews, or the developer's desire to optimize.
- It refines existing functionality rather than adding entirely new modules
- Performed on a running, bug-free system.

4.8. Testing Strategies

Black box testing:

- Use test data for which results already calculated & compare result from program with expected results
- Testing only considers input and output and the code is viewed as being in a 'black box'

White box testing:

- Examine each line of code for correct logic and accuracy.
- May record value of variables after each line of code
- Every possible condition must be tested

Stub testing:

- Stubs are computer programs that act as temporary replacement for a called module and give the same output as the actual product or software.
- Important when code is not completed however must be tested so modules are replaced by stubs

Dry run testing:

- A process where code is manually traced, without any software used
- The value of a variable is manually followed to check whether it is used and updated as expected

- Used to identify logic errors, but not execution errors

Walkthrough testing:

- A test where the code is reviewed carefully by the developer's peers, managers, team members, etc.
- It is used to gather useful feedback to further develop the code.

Integration testing:

- Taking modules that have been tested on individually and testing on them combined together
- This method allows all the code snippets to integrate with each other, making the program work.

Alpha testing:

- This is the testing done on software 'in-house', meaning it is done by the developers
- Basically another term for 'first round of testing'

Beta testing:

- This is the testing done on the software by beta users, who use the program and report any problems back to the developer.
- Basically another term for 'second round of testing'

Acceptance testing:

- A test carried out by the intended users of the system: the people who requested the software.
- The purpose is to check that the software performs exactly as required.
- The acceptance criteria should completely be satisfied for the program to be released.

NOTES